

Q1. Should I pre-process my input assembly fasta file?

A1. Potentially. Below are two places where we suggest pre-processing your assembly, if needed.

- a. Your input genome assembly should have the most simple fasta alphabet (T,C,A,G, N for missing base-pairs, softmasked or unmasked). Some fasta files contain additional letters, representing uncertain basepair calls (e.g., R = A or G). Please replace these uncertain calls to an "N".
- b. Genome alignment tools (e.g., hisat2), TOGA, and Braker will all pre-process complex fasta headings slightly differently. For example, hisat2 removes everything after the first space, while Braker replaces spaces with underscores. We recommend removing everything past the first space in each fasta header, and making sure that the fasta header is still unique between each contig.

We clean our reference fasta files for these potential issues in setup. The same code will work for your input fasta file. Below uses mouse GRC39 as an example. This code does not change masking so your inputted assembly will remain softmasked.

...

```
fa=GCF_000001635.27_GRCm39_genomic.fna
prefix=GCF_000001635.27_GRCm39_genomic
```

```
echo "Clean reference fasta file"
sed '/^>/ s/ .*//' $fa | sed 's/[ryswkmbdhv]/N/gi' > $prefix.cleanhead.fa
```

```
# Index FASTA file
samtools faidx $prefix.cleanhead.fa
```

```
# Extract unique contig names from GFF
awk '$1 !~ /^#/ {print $1}' $prefix.gffread.gff | sort -u > contigs_with_gene.txt
```

```
# Extract contigs
xargs samtools faidx $prefix.cleanhead.fa < contigs_with_gene.txt > prefix.clean.fa
```

```
rm $prefix.cleanhead.fa
...
```

Q2. Do I have to use EarlGrey to mask my genome assembly?

A2. No, you can input any softmasked assembly and any bed file designation repeat annotations. You can use the masked assembly if your genome requires additional or different repeat annotation steps. In the Snakemake, put the masked assembly fasta in both "assemblyFile" and "MaskedAssemblyFile".

Q3. The snakemake file asks for too many memory requirements. Do I need a better computational cluster?

A3. Not necessarily. We ask for the runtime and memory that we found sufficient to annotate a rodent genome (with RNA-seq data) as the default. We do not have a parameter requiring memory for every rule to avoid a massive number of parameters in the config. However, the “threads” and “params” in each rule designate how much memory/runtime each rule will ask for in annotations. Below is an example of the mask_assembly rule. If the number of threads is too much, the walltime is too high, or you need extra options to submit jobs. The snakemake file can be edited directly. Just make sure to save a copy of the new snakefile somewhere so you don't overwrite your fixes if you ever clone GenAnT again.

...

rule mask_assembly:

input:

config["outDir"]+"/assembly/assembly.fa"

output:

config["outDir"]+"/assembly/assembly.softmasked.fa"

threads:

36 # Request 16 threads for this rule

params:

memory_per_thread="8G",

extra_cluster_opt="",

walltime="150:00:00"

shell:

"""

scripts/run_earl_grey.sh {config[outDir]} {config[externalDir]} {config[species]}
{config[MaskedAssemblyFile]} {config[MaskedAssemblyAnnotation]} {config[dfamDB]}

"""

...

Q4. Are you going to add additional methods?

A4. Yes. As long as people are using this tutorial, we will continue to update annotation methods moving into the future. Currently, the tools we are testing and will subsequently include, assuming there are no issues, are liftOn (<https://github.com/Kuanhao-Chao/LiftOn>), EviAnn (https://github.com/alekseyzimin/EviAnn_release), and Tiberius (<https://github.com/Gaius-Augustus/Tiberius>).

Q5. Will you make a single container for GenAnT?

A5. We are currently moving to have installation be a list of containers rather than as a single container for a few reasons. Firstly, TOGA would require substantial rewrites before being able to be compiled into a Docker container. Secondly, many tools install very well with precompiled binaries (e.g., stringtie, InterProScan). Thirdly, genome annotation tools are consistently evolving due to improved deep learning models and the need to annotate T2T assemblies at scale. As such, the specific components of GenAnT are subject to change until these tools are benchmarked.

Q6. Gene models transferred from TOGA and LiftOff transfer the geneID when the reference annotation comes from a source that is not RefSeq (e.g., ENSEMBL, custom gff, etc.).

A6. We have tried to fix this in our automatic pipelines and found that a catch-all script to ensure gene symbols instead of gene IDs would be transferred was too error-prone due to the diversity of inputted GFF files. Instead, we have provided scripts in “utils” to replace gene IDs with gene symbols in the gene_symbol.tsv file generated in step 5 before remaking the final gff file. This utils script takes about 10 minutes to run on a desktop computer. It also lets you add gene symbols from sources that are not automatically run by GenAnT (e.g., OMA).

Q7. Is ISO-seq the only long-read RNA-seq method compatible with GenAnT?

A7. They are not. Long-read RNA-seq gene models are generated with stringtie -mix, stringtie -L, or BrakerV3 (which also uses stringtie -L). Broadly, these methods assume that a confidently aligned read is a gene model. As such, any long read RNA-seq .bam file will work.

Q8. How large is the GenAnT output?

A8. Large, as it keeps intermediate files and copies assembly bam file information into /assembly, /RNAseq_alignment and /ISOseq_alignment. We typically see about a 300Gb output file when annotating a rodent genome with 10-20 bam files for RNA-seq and ISO-seq. If possible, we suggest running GenAnT in scratch and copying over files necessary for downstream analysis. Examples of these files are: the annotation.gff, gene_symbol.tsv, comparative genomics statistics from orthofinder and toga if valuable, a the mikado score.tsv and metrics.tsv tables to evaluate transcripts. These files are detailed in the stepwise tutorial.

Q9. My run failed. Is there anything I need to do before rerunning?

Q9. Possibly. Here are some places to consider. We go through this in detail in the “clean_GenAnT_after_failure.md” on the github. Some broad points are here:

- EarlGrey picks up where you left off even if the EarlGrey step failed.
- The snakemake tries to pick up at the rule that previously succeeded.
- The snakemake has locked your previous run and not unlocked the required directories (can occur if ended prematurely). Delete the .snakemake file in the folder to resolve this issue.
- Cactus will automatically fail if you do not delete the “jobstore” directory, so if cactus failed you will have to delete “jobstore” in cactus_alnr1 and cactus_alnr2. If cactus runs successfully it should clean jobstore on its own.
- Depending on how the job fails, GenAnT may make an empty file for one of the GFFs in step 2. For example, if it falsely didn’t detect RNA-seq data (e.g., there is a typo to your path to the bam files in your input), it may “touch” the stringtie.gffread.gff, [braker.sr.gffread.gff](#), and [braker.lr.gffread.gff](#) into /transcript_selection. As such, GenAnT may run in non-RNA-seq mode. Accordingly, it’s good to check that /transcript_selection has the expected gff files to integrate before restarting a run, or even after what looks like a successful run. You can check this with `ls -l /transcript_selection/*gff`. A “touched” gff file will have a filesize of 0 bytes.

-

Q10. Stringtie was running quickly for many tissues and has been stuck on a single tissue forever. Any ideas what's going wrong?

A10. We (amongst some others) have come across this issue, particularly for ISO-seq data. This error was resolved in StringTie/2.2.1 but is seemingly reintroduced (<https://github.com/gpertea/stringtie/issues/476>). We will update with the fix when it comes.